Proyecto Kronar: division del trabajo

Haskell (parte 1) + Julia (parte 2). Grupo de 4 personas. Este documento dice, para cada persona, **que funciones escribe, con que firma, que debe cumplir, que pruebas y documentos entrega, de quien depende y cuando esta listo**.

1. Vista general

	Rol	Haskell (`Kronar.hs`)	Julia (`integracion.jl`)	Pruebas y README	Horas
P1	Entrada, meta y lectura del JSON	Validacion del tablero, Regla del Vacio, tablero editable. Mensajes de error.	Leer y validar `resultado.json` sin caer.	Tablero A + t1 a mano. README: instalacion y ejecucion.	~9 h
P2	Caminos, optimo y verificacion	Generacion recursiva de caminos, conteo, desempates, eleccion del optimo.	Reglas en Julia y verificacion del puntaje del JSON; porcentaje por casilla.	Tablero B (empate) + t2 a mano.	~10 h
P3	Reglas y estadistica	Puntaje bruto, penalizacion Zafiro, bono Eter, puntaje final.	Estadisticas de casillas en juego y grafico de barras en consola.	Tablero C (Zafiro y empate) + t3 a mano.	~10 h
P4	Salida, integracion y reporte	JSON a mano, `resultado.json` con t1, resumen en consola, `main`, unir `Kronar.hs`.	Reporte comun, `main` y union en `integracion.jl`.	`Pruebas.hs` final, README final y prueba de extremo a extremo.	~10 h

Las horas son una estimacion orientativa para equilibrar la carga; lo importante es que las cuatro personas tocan Haskell, Julia y documentacion.

2. Acuerdos obligatorios antes de escribir codigo (reunion de 1 hora)

- **Un repositorio** con carpeta `haskell/` y `julia/`. Cada persona trabaja en su rama o en su seccion y no edita las secciones de otros.
- **Un solo archivo entregable por lenguaje.** Durante el desarrollo cada persona puede tener su propio archivo, pero al final P4 une todo en `Kronar.hs` y en `integracion.jl`, separando las secciones con banners de comentario (`-- === SECCION 2: CAMINOS (P2) ===`).
- **Indices:** en el JSON y en Haskell las posiciones empiezan en **0**. Los reportes para personas numeran desde **1**. En Julia los vectores empiezan en 1: convertir con `+1` en un unico punto (una funcion auxiliar) y documentarlo.
- **Haskell:** solo lo que viene con GHC (Prelude, `Data.List`, `System.Environment`, `System.Exit`). Prohibidas librerias externas. Toda definicion con firma de tipo escrita.
- **Estilo:** funciones pequenas con una sola responsabilidad, comentario en toda logica no obvia, nombres en espanol, igual que las claves del JSON.

Tipos compartidos de Haskell (los define P3 el dia 1; todos los usan tal cual)

```
type Tablero = [Int]
type Camino  = [Int]               -- posiciones desde 0, ej. [0,1,3]

data Puntaje = Puntaje
  { pBruto  :: Int
  , pZafiro :: Int                 -- penalizacion ya como numero positivo (0, 5, 10...)
  , pEter   :: Int                 -- 0 o 10
  , pFinal  :: Int
  } deriving (Show, Eq)

data Resultado = Resultado
  { rTablero   :: Tablero
  , rMeta      :: Int              -- desde 0
  , rVacio     :: Bool
  , rCamino    :: Camino
  , rPuntaje   :: Puntaje
  , rEvaluados :: Int
  } deriving (Show, Eq)
```

Firmas puente entre personas (se acuerdan hoy; cada dueno las implementa)

Firma	Dueno	Usa	Contrato
`calcularMeta :: Tablero -> Int`	P1	P2, P4	Indice (desde 0) de la meta tras la Regla del Vacio.
`generarCaminos :: Int -> [Camino]`	P2	P4	Todos los caminos de 0 a la meta.
`puntuarCamino :: Tablero -> Camino -> Puntaje`	P3	P2, P4	Puntaje completo de un camino. **P3 entrega una version provisoria el dia 1** para no bloquear.
`elegirOptimo :: [(Camino, Puntaje)] -> (Camino, Puntaje)`	P2	P4	Mejor camino con desempates.
`resultadoAJson :: Resultado -> String`	P4	todos	Texto exacto de `resultado.json`.

Tipos compartidos de Julia (los define P1 el dia 1; se conservan los indices base 0 del JSON dentro del registro)

```
struct Resultado
    tablero::Vector{Int}
    casilla_meta::Int              # base 0, tal cual el JSON
    regla_vacio_aplicada::Bool
    camino_optimo::Vector{Int}     # base 0, tal cual el JSON
    puntaje_bruto::Int
    penalizacion_zafiro::Int
    bono_eter::Int
    puntaje_final::Int
    caminos_evaluados::Int
end

struct ErrorLectura
    mensaje::String                # texto legible para el usuario
end
```

PERSONA 1

Entrada, meta y lectura del JSON

Validar el tablero, aplicar la Regla del Vacio y leer resultado.json en Julia sin caerse.

Carga estimada **~9 horas**

A. Haskell

Funcion	Firma	Que debe hacer
`tableroPorDefecto`	`:: Tablero`	**Unico lugar editable**, con banner de comentario bien visible. Valor inicial: t1 = [3,-2,-1,4,2].
`leerTablero`	`:: [String] -> Either String Tablero`	Si hay argumentos (`runghc Kronar.hs 3 -2 -1 4 2`) los convierte a enteros con `reads`; si no hay, usa el tablero por defecto. Si un argumento no es entero, devuelve `Left` con el valor que fallo.
`validarTablero`	`:: Tablero -> Either String Tablero`	Rechaza con mensaje propio cada caso: menos de 2 casillas, mas de 15, o todas en 0. El mensaje dice *que* esta mal y *cuanto* (ej. 'Se recibieron 17 casillas; el maximo es 15').
`calcularMeta`	`:: Tablero -> Int`	**Regla del Vacio.** Retrocede desde el final hasta la ultima casilla distinta de 0. Devuelve indice desde 0. Se llama solo con tablero ya validado.
`vacioAplicado`	`:: Tablero -> Int -> Bool`	True si la meta no es la ultima casilla (`meta /= length t - 1`).

B. Julia

Funcion	Firma	Que debe hacer
`struct Resultado / ErrorLectura`	(ver seccion 2)	Definirlos el dia 1 y avisar al grupo.
`leer_resultado`	`(ruta)::Union{Resultado,ErrorLectura}`	Si el archivo no existe: mensaje claro. Si no se puede parsear (truncado, vacio, basura): 'archivo danado' con motivo. Todo dentro de `try/catch`: nunca lanza excepcion.
`validar_estructura`	`(datos)::Union{Resultado,ErrorLectura}`	Comprueba las **9 claves** con nombre exacto y el tipo de cada una (listas de enteros, entero, booleano). **Trampa:** en Julia `Bool <: Integer`, asi que revisar Bool *antes* de Int. Verificar tambien rangos: meta y camino dentro de `0:length(tablero)-1`, penalizacion >= 0, bono en {0,10}.
`mensaje de error`	texto	Siempre nombrar la clave y decir 'se esperaba X, se encontro Y'.

C. Pruebas y documentacion

- **Parser JSON:** usar `JSON.jl` (`] add JSON`) y anotarlo en el README, **o** escribir un parser pequeno (el formato es plano y fijo). Confirmar con el profesor si se permiten paquetes en Julia; si hay duda, parser propio.
- **Archivos de prueba de error** en `julia/errores/`: sin una clave, con un tipo equivocado, truncado, vacio y nombre inexistente. Son la evidencia del criterio 'sin caerse'.
- **Pruebas.hs:** Tablero A `[4,-1,2,0,0]` (prueba la Regla del Vacio) con tabla de todos los caminos y justificacion a mano. Ademas, resolver **t1** a mano (7 caminos) para contrastar con el JSON generado.

- **README:** secciones 'Requisitos e instalacion' (versiones de GHC y Julia, paquete JSON) y 'Como ejecutar Haskell' (con y sin argumentos, ejemplos de errores esperados).

Depende de / entrega a

- Recibe de P4 el `resultado.json` real (mientras tanto, usa el ejemplo de la seccion 4.2 del enunciado).
- Entrega a P2, P3 y P4 el tipo `Resultado` y `leer_resultado` el dia 1-2.
- Entrega a P2 y P4 `calcularMeta` el dia 1.

Esta listo cuando...

- Un tablero de 1 casilla, de 16 casillas o de solo ceros da un mensaje distinto y **no se crea ningun archivo**.
- Los 5 archivos danados de `julia/errores/` producen mensajes comprensibles y el programa termina limpio.
- El tablero A y t1 tienen su tabla de caminos resuelta a mano y coincide con la salida del programa.

PERSONA 2

Caminos, optimo y verificacion

Generar todos los caminos con recursion, elegir el optimo y verificar el puntaje del JSON en Julia.

Carga estimada **~10 horas**

A. Haskell

Funcion	Firma	Que debe hacer
`saltosValidos`	`:: [Int]`	`[1,2,3]`. Una constante evita numeros magicos.
`caminosDesde`	`:: Int -> Int -> [Camino]`	**Recursion obligatoria.** Parametros: meta y posicion actual. Caso base: actual == meta devuelve `[[meta]]`. Caso recursivo: para cada salto s con `actual+s <= meta`, antepone actual a cada camino de `caminosDesde meta (actual+s)`. Nunca se pasa de la meta.
`generarCaminos`	`:: Int -> [Camino]`	Llama a `caminosDesde meta 0`. El conteo de `caminos_evaluados` es el `length` de esta lista completa (no de una filtrada).
`compararCandidatos`	`:: (Camino,Puntaje) -> (Camino,Puntaje) -> Ordering`	Orden total: 1) mayor `pFinal`; 2) menos saltos (camino mas corto); 3) menor lexicografico (`compare` sobre listas ya lo hace). **Trampa:** `maximumBy` devuelve el *ultimo* de los empatados; como el criterio 3 desempata siempre, no queda ambiguedad si lo implementas completo.
`elegirOptimo`	`:: [(Camino,Puntaje)] -> (Camino,Puntaje)`	Aplica el comparador sobre la lista completa. La lista nunca esta vacia en un tablero valido (siempre existe el camino de saltos de 1).

B. Julia

Funcion	Firma	Que debe hacer
`a_base1`	`(v::Vector{Int})`	Convierte posiciones del JSON (base 0) a indices de Julia (`v .+ 1`). Unico punto de conversion para todo el equipo.
`puntaje_bruto`	`(tablero, camino)::Int`	Suma `tablero[a_base1(camino)]`.
`penalizacion_zafiro`	`(tablero, camino)::Int`	5 puntos por cada **par consecutivo del camino** con ambos valores < 0 (tres negativos seguidos = 10). El 0 no es negativo. Consecutivo es en el camino, no en el tablero.
`bono_eter`	`(bruto::Int)::Int`	10 si el bruto es par (incluye 0 y negativos: `iseven`); si no, 0. Se calcula sobre el bruto, antes de la penalizacion.
`calcular_puntaje`	`(tablero, camino)::NamedTuple`	Devuelve `(bruto, zafiro, eter, final)` con `final = bruto - zafiro + eter`.
`verificar_puntaje`	`(res::Resultado)::Bool`	Recalcula con el camino del JSON y compara los 4 campos. Imprime OK o, si difiere, 'el JSON dice X, Julia calcula Y, diferencia Z'. **No copia valores del JSON** (exigencia 5.1.5).
`porcentaje_por_casilla`	`(res)`	Para cada casilla visitada: valor / bruto * 100, con numero de casilla desde 1. **Caso especial:** si el bruto es 0, avisar en vez de dividir por cero. Los porcentajes pueden ser negativos o superar 100: es normal.

C. Pruebas y documentacion

- **Pruebas.hs:** Tablero B `[-5,7,4,0,8]`. Empate entre caminos con **distinto numero de saltos**, un 0 en medio del camino y inicio negativo. Tabla de los 7 caminos y justificacion a mano del desempate. Resolver tambien **t2** a mano.
- **Prueba cruzada:** ejecutar `calcular_puntaje` de Julia contra `puntuarCamino` de Haskell en t1, t2, t3 y los 3 tableros propios. Los resultados deben coincidir exactamente.
- **README:** explicar en 5 lineas como se genera la lista de caminos y como se desempata.

Depende de / entrega a

- Depende de P3 para `puntuarCamino` (version provisoria el dia 1) y de P1 para `calcularMeta` y el tipo `Resultado` de Julia.
- Entrega a P4 `generarCaminos` y `elegirOptimo` el dia 2, y `verificar_puntaje` y `porcentaje_por_casilla` el dia 4.

Esta listo cuando...

- Para t1 el programa evalua exactamente los caminos que salen a mano en la tabla (7) y para una meta de indice m no genera ningun camino que pase de m.
- `generarCaminos` esta escrita con recursion (sin filtrar combinaciones).
- Con un JSON alterado a mano (puntaje_final cambiado), `verificar_puntaje` muestra la diferencia correcta.

PERSONA 3

Reglas de puntaje y estadistica

Implementar las tres reglas en Haskell y el analisis estadistico con grafico de barras en Julia.

Carga estimada **~10 horas**

A. Haskell

Funcion	Firma	Que debe hacer
`(tipos compartidos)`	ver seccion 2	**Dia 1:** escribir `Tablero`, `Camino` y `Puntaje` y una version provisoria de `puntuarCamino` para desbloquear a P2.
`puntajeBruto`	`:: Tablero -> Camino -> Int`	Suma el valor de cada casilla del camino. La casilla inicial siempre cuenta; las saltadas no.
`penalizacionZafiro`	`:: Tablero -> Camino -> Int`	Toma los valores del camino y los recorre de a pares consecutivos (`zip vs (tail vs)`); cada par con ambos < 0 suma 5. Tres negativos seguidos suman 10. Cero no es negativo.
`bonoEter`	`:: Int -> Int`	10 si el bruto es par (`even` sirve con 0 y negativos), si no 0.
`puntuarCamino`	`:: Tablero -> Camino -> Puntaje`	Combina las tres: `pFinal = bruto - zafiro + eter`. Un comentario que explique las reglas a quien lea el codigo.

B. Julia

Funcion	Firma	Que debe hacer
`casillas_en_juego`	`(res)::Vector{Int}`	`tablero[1:casilla_meta+1]`. Las casillas posteriores a la meta no entran en **ninguna** estadistica.
`estadisticas`	`(v::Vector{Int})`	Media, mediana, desviacion estandar (`using Statistics`), maximo y minimo. Decidir y documentar: desviacion muestral (n-1, por defecto) o poblacional (`corrected=false`); y que pasa si el maximo o minimo se repite (indicar la primera o todas las casillas).
`conteo_signos`	`(v)::NamedTuple`	Cuantas piedras positivas, negativas y vacias, solo en juego.
`imprimir_estadisticas`	`(res)`	Imprime todo, indicando **en que casilla (desde 1)** estan el maximo y el minimo. Ej: 'Maximo: 5 en la casilla 4'.
`estado_casilla`	`(i, camino, meta)::Symbol`	Devuelve `:visitada`, `:saltada` o `:fuera_de_juego` (i mayor que la meta).
`grafico_barras`	`(res)`	Una linea por casilla (todas, para mostrar las fuera de juego): `casilla 1 3 ▓▓▓▓▓▓ visitada`. Largo = valor absoluto; positivos con `▓`, negativos con `▒` (distintos a simple vista); 0 con barra vacia. Alinear columnas con `lpad/rpad`. Texto, sin ventanas.

C. Pruebas y documentacion

- **Pruebas.hs:** Tablero C `[9,4,-3,-3,-6]`: la penalizacion Zafiro aparece en el camino optimo (con dos pares seguidos en algun otro camino), el bono Eter se activa y hay un **empate decidido por orden lexicografico**. Tabla de los 7 caminos y justificacion. Resolver tambien **t3** a mano (13 caminos).
- **Nota del grafico:** probarlo con t1, t2 y t3 (ver que las casillas fuera de juego de t2 aparezcan marcadas) y con un tablero con valores grandes (escalar la barra si pasa de 20 caracteres).
- **README:** sección sobre las decisiones estadisticas (n-1 vs n, empates de maximo/minimo) y una muestra del grafico.

Depende de / entrega a

- Entrega a P2 la version provisoria de `puntuarCamino` el dia 1 (la definitiva el dia 2).
- Depende de P1 para el tipo `Resultado` y `leer_resultado` en Julia.
- Entrega a P4 `imprimir_estadisticas` y `grafico_barras` el dia 4.

Esta listo cuando...

- Con t1, la penalizacion, el bono y el puntaje final de Haskell coinciden con los calculados a mano para cada camino.
- El grafico muestra con claridad visitada / saltada / fuera de juego y los negativos se ven distintos de los positivos.
- Ninguna estadistica incluye casillas posteriores a la meta.

PERSONA 4

Salida, integracion y reporte

Armar el JSON, el programa principal y el reporte, unir los archivos y verificar el flujo completo.

Carga estimada **~10 horas**

A. Haskell

Funcion	Firma	Que debe hacer
`listaAJson`	`:: [Int] -> String`	Escribe `[2, -1, 3]`. Numeros negativos con `show` funcionan bien.
`boolAJson`	`:: Bool -> String`	**Trampa:** `show True` da 'True'; el JSON exige `true`/`false` en minuscula.
`resultadoAJson`	`:: Resultado -> String`	Las 9 claves exactamente con estos nombres y en este orden: `tablero`, `casilla_meta`, `regla_vacio_aplicada`, `camino_optimo`, `puntaje_bruto`, `penalizacion_zafiro`, `bono_eter`, `puntaje_final`, `caminos_evaluados`. Sin librerias.
`escribirResultado`	`:: FilePath -> Resultado -> IO ()`	`writeFile "resultado.json"`.
`resolver`	`:: Tablero -> Resultado`	Orquesta: meta (P1) → caminos (P2) → puntuar cada uno (P3) → optimo (P2). Arma el `Resultado` con el conteo de caminos.
`resumenConsola`	`:: Resultado -> String`	Tablero, meta, camino optimo, desglose (bruto, penalizacion, bono, final) y cuantos caminos se evaluaron.
`main`	`:: IO ()`	Lee y valida el tablero (P1). Si falla: imprime el mensaje, sale con codigo de error y **no crea archivo**. Si funciona: resuelve, muestra el resumen y escribe `resultado.json`.

B. Julia

Funcion	Firma	Que debe hacer
`formatear_ruta`	`(camino)::String`	Convierte posiciones (base 0) en texto de ruta: '1, 2 y 4'. Cubrir 2 casillas ('1 y 4') y 1 casilla.
`reporte_comun`	`(res)`	Tablero con casillas numeradas desde 1, meta (casilla N) con el aviso de si la Regla del Vacio se aplico, ruta, y desglose de puntaje (bruto, penalizacion, bono, final).
`main`	`()`	Recibe la ruta del archivo (por defecto `resultado.json`). Orden de salida: lectura (P1) → reporte comun → estadisticas (P3) → grafico (P3) → verificacion (P2) → porcentajes (P2). Si la lectura falla, imprime el mensaje y termina limpio.

C. Pruebas y documentacion

- **Union final:** montar `Kronar.hs` e `integracion.jl` con secciones marcadas. Compilar con `ghc -Wall -Wmissing-signatures` y revisar que no queden advertencias de firma ni imports externos.
- **resultado.json oficial:** generarlo con **t1** y verificar con un lector de JSON que es valido.

- **Pruebas.hs:** reunir los 3 tableros propios (A de P1, B de P2, C de P3) y t1, t2, t3 con sus tablas. Verificar que en conjunto cubran las tres reglas y al menos un empate.
- **README.md:** unificar las secciones de todos, agregar muestras reales de pantalla de ambos programas y el orden de ejecucion (primero Haskell, despues Julia).
- **Prueba de aceptacion:** clonar el repositorio en una carpeta limpia, seguir el README paso a paso y confirmar que todo corre. Comparar Haskell y Julia en todos los tableros.

Depende de / entrega a

- Depende de P1 (`leerTablero`, `validarTablero`, `calcularMeta`), P2 (`generarCaminos`, `elegirOptimo`) y P3 (`puntuarCamino`).
- Entrega el `resultado.json` real a P1, P2 y P3 el dia 3 para que prueben Julia con datos reales.
- Hace la union final el dia 6 y la prueba desde cero el dia 7.

Esta listo cuando...

- El JSON generado coincide clave por clave, en orden y con los tipos del enunciado (booleano en minuscula, listas de enteros).
- Haskell con un tablero invalido imprime el error y **no** deja `resultado.json`.
- Cualquier miembro del grupo ejecuta ambos programas desde cero siguiendo solo el README.

3. Cronograma sugerido (7 dias; comprimir manteniendo el orden si la fecha de entrega es menor)

Fase	Que se hace	Quien	Resultado esperado
Fase 0 Dia 1	Reunion de 1 h: repositorio, tipos compartidos, firmas puente, convenciones.	Todos	Repositorio creado; secciones y firmas acordadas.
Fase 1 Dias 1-2	Cada persona escribe su seccion de Haskell. P3 entrega `puntuarCamino` provisoria el dia 1. P1 define los tipos de Julia el dia 1.	P1, P2, P3	Cada funcion compila y se prueba sola con t1.
Fase 2 Dias 3-4	P4 une el Haskell, genera `resultado.json` con t1. Mientras tanto, P1, P2 y P3 trabajan Julia con el JSON de ejemplo del enunciado.	P4 (Haskell), P1-P3 (Julia)	Haskell completo y `resultado.json` real disponible.
Fase 3 Dias 4-6	Julia: lectura (P1), reglas y verificacion (P2), estadistica y grafico (P3), reporte (P4). En paralelo, cada uno resuelve a mano su tablero y su tablero oficial.	Todos	Cada seccion de Julia lista; tablas a mano terminadas.
Fase 4 Dia 6	Revision cruzada: P1 revisa a P2 y P2 a P1; P3 revisa a P4 y P4 a P3. P4 une `integracion.jl` y `Pruebas.hs`.	Todos	Un archivo por lenguaje, sin advertencias.
Fase 5 Dia 7	README final con muestras de pantalla. Prueba desde carpeta limpia. Entrega.	P4 lidera, todos revisan	Checklist de entrega completo.

4. Mapa de la rubrica (100 puntos) a responsables

Criterio	Pts	Responsable	Revisa
Genera todos los caminos de forma recursiva y respeta la meta	15	P2	P1
Las tres reglas funcionan correctamente	15	P3	P4
Camino optimo y desempates	10	P2	P1
`resultado.json` valido y con el formato exacto	10	P4	P3
Valida el tablero y da mensajes claros	5	P1	P2
Parte 2: lectura, validacion y reporte comun	10	P1 (lectura y validacion) + P4 (reporte)	P3
Parte 2: tarea especifica de Julia	20	P2 (calculo y verificacion) + P3 (estadistica y grafico)	P4
Pruebas propias bien justificadas	10	Todos (un tablero cada uno); P4 une	Todos
README claro y funcional	5	P1 (instalacion) + P4 (muestras y union)	P2

5. Trampas frecuentes (revisarlas antes de entregar)

- **Indices:** el JSON y Haskell usan base 0; los reportes para personas y los vectores de Julia usan base 1. Un camino `[0,1,3]` se narra como 'casillas 1, 2 y 4'.
- **Eter** se calcula sobre el puntaje *bruto*, antes de restar la penalizacion; 0 y los pares negativos cuentan como pares.
- **Zafiro** mira pares consecutivos *del camino*, no del tablero; si Dravek salta de la casilla 2 (-1) a la 4 (-3), se aplica. Tres negativos seguidos restan 10.
- **Casilla inicial** siempre cuenta; si es negativa tambien entra en el calculo Zafiro con la siguiente.
- **Casillas despues de la meta** no cuentan en nada: ni caminos, ni estadisticas, ni porcentajes. En el grafico aparecen marcadas como fuera de juego.

- **Haskell:** `show True` produce 'True', no 'true'. Y `maximumBy` devuelve el ultimo de los empatados: usar un comparador total.
- **Julia:** `true isa Integer` es verdadero: validar Bool antes que Int. `std` divide por n-1 por defecto. Dividir por el bruto cuando vale 0 da error.
- **No crear archivo** si el tablero es invalido. Cada definicion de Haskell con su firma de tipo.

6. Checklist de entrega

Archivo	Debe contener	Responsable
`Kronar.hs`	Haskell completo, sin librerias externas, todo con firma y comentarios.	P4 une; todos revisan
`resultado.json`	Generado con t1, claves exactas, en orden y con los tipos correctos.	P4
`integracion.jl`	Lectura, validacion, reporte, estadistica, grafico, verificacion y porcentajes.	P4 une; P1, P2, P3 aportan
`Pruebas.hs`	Al menos 3 tableros propios, con tabla de caminos y resultado justificado a mano, mas t1, t2, t3. Cubren las tres reglas y un empate.	P1, P2, P3; P4 une
`README.md`	Que instalar, como ejecutar ambos programas en orden, muestras de pantalla.	P1 y P4

7. Tableros propios sugeridos para Pruebas.hs

Estos tableros se eligieron para que, en conjunto, cubran la Regla del Vacio, el Eter, el Zafiro y los dos tipos de empate. Cada persona debe construir **a mano** la tabla de todos los caminos de su tablero (como en la seccion 3 del enunciado) y justificar el resultado; despues se compara contra el programa. Valores de autocomprobacion: caminos evaluados y puntaje final.

	Tablero	Que demuestra	Autocomprobacion
P1	`[4, -1, 2, 0, 0]`	Regla del Vacio (la meta no es la ultima casilla). Eter.	2 caminos; final 16
P2	`[-5, 7, 4, 0, 8]`	Empate con distinto numero de saltos (gana el de menos saltos). Un 0 en medio del camino. Inicio negativo.	7 caminos; final 24
P3	`[9, 4, -3, -3, -6]`	Zafiro en el camino optimo, Eter y empate decidido por orden lexicografico.	7 caminos; final 9

Los tableros oficiales t1, t2 y t3 no traen resultados a proposito: su resolucion a mano y su justificacion son parte de la tarea.